

Rapport de projet : EventHub

Plateforme de gestion d'événements du Dakar Institute of Technology

Examen pratique DevOps, Master 1 Intelligence Artificielle Période : 5 août 2026 → 19 août 2026

Auteur : **Makilou Cherif Tine**, projet réalisé individuellement (conception, développement backend, frontend et DevOps)

1. Introduction

1.1 Contexte

Le Dakar Institute of Technology organise chaque année de nombreux événements académiques et culturels : conférences, ateliers, séminaires, journées portes ouvertes. Leur gestion reposait jusqu'ici sur un assemblage d'outils non connectés, formulaires Google pour les inscriptions, feuilles Excel pour le suivi, emails pour la communication.

Ce fonctionnement présente quatre limites majeures :

- 1. **Fragmentation de l'information.** La même inscription existe dans un formulaire, une feuille de calcul et une boîte mail, sans source de vérité.
- 2. **Aucun suivi en temps réel.** Le nombre de places restantes n'est connu qu'après consolidation manuelle, ce qui provoque des sur-inscriptions.
- 3. **Absence de tableau de bord.** Les organisateurs n'ont pas de vision consolidée du taux de remplissage de leurs événements.
- 4. **Communication difficile.** Les coordonnées des participants sont dispersées et souvent incomplètes.

1.2 Objectif du projet

EventHub est une plateforme web unifiée qui centralise la création d'événements, la gestion des participants et le suivi des inscriptions, avec un contrôle de capacité en temps réel et un tableau de bord statistique.

Au-delà du produit, l'objectif pédagogique est de mettre en pratique une chaîne DevOps complète : architecture microservices, conteneurisation Docker, intégration et déploiement continus avec GitHub Actions, et gestion de projet avec Git et GitHub.

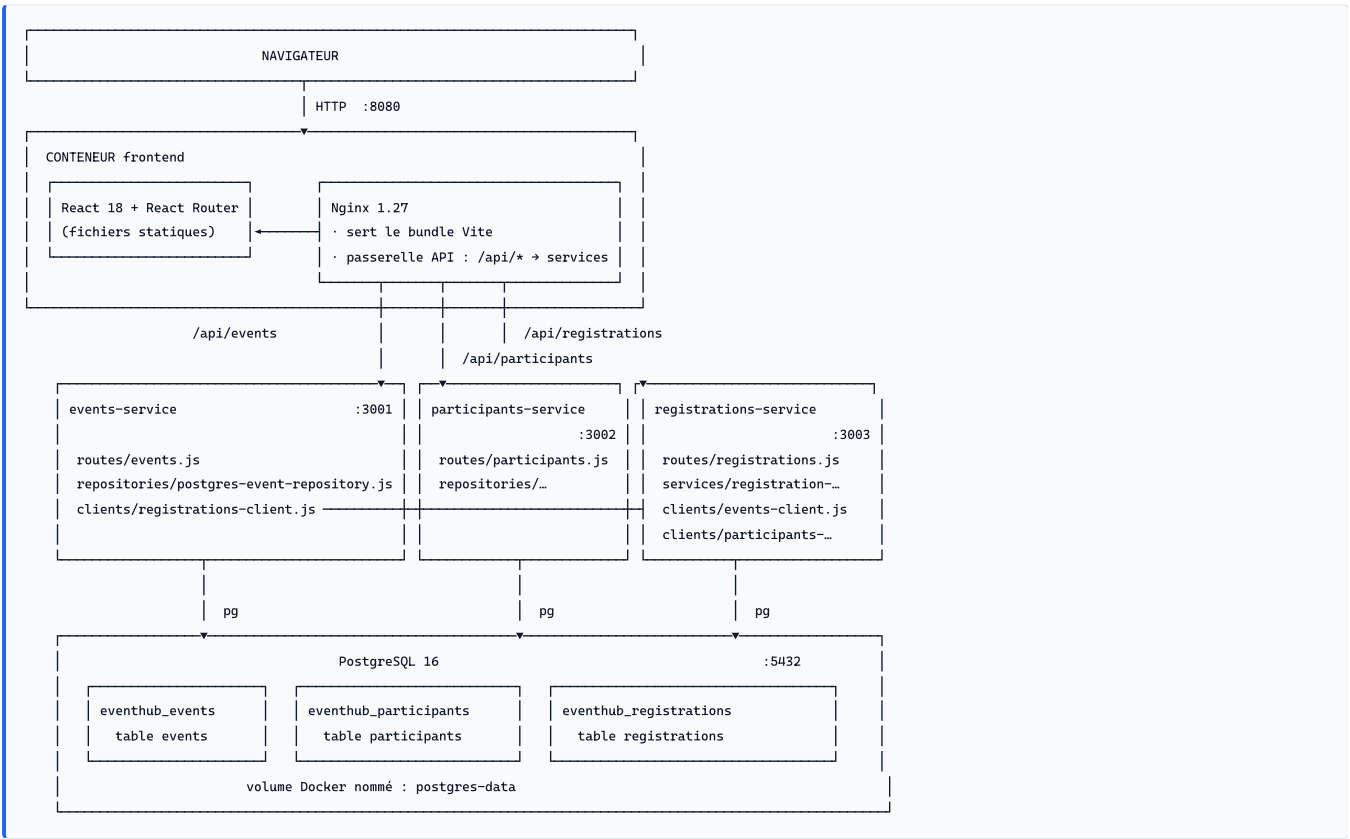
1.3 Périmètre fonctionnel livré

Domaine	Fonctionnalités
Événements	Création, modification, suppression, liste filtrable (date, lieu, recherche plein texte), détail, places restantes
Participants	Création de compte (nom, email, téléphone, type), modification, suppression, liste, recherche par email ou nom
Inscriptions	Inscription à un événement, annulation, inscriptions d'un événement, événements d'un participant, statistiques, détection des places disponibles
Transverse	Tableau de bord consolidé, état de santé des services, jeu de données de démonstration

2. Architecture

2.1 Vue d'ensemble

L'application suit une architecture **microservices** : trois services backend autonomes, chacun responsable d'un domaine métier, un frontend React, et une base de données relationnelle PostgreSQL découpée en trois bases logiques.



2.2 Découpage en services

Le découpage suit les **frontières du domaine métier**, et non les couches techniques :

Service	Responsabilité	Donnée dont il est propriétaire
events-service	Cycle de vie des événements et leur capacité	table events
participants-service	Identité et profil des participants	table participants
registrations-service	Lien entre un participant et un événement	table registrations

2.3 Une base de données par service

Chaque microservice possède sa **propre base de données**, et il est le seul à y accéder. Aucune requête SQL ne traverse la frontière d'un service.

Ce choix a un coût (il n'existe pas de clé étrangère entre **registrations** et **events**), mais il apporte l'indépendance qui justifie l'architecture microservices :

- le schéma d'un service peut évoluer sans coordonner les autres équipes ;
- un service peut être déployé, redémarré ou migré séparément ;
- une panne de schéma reste confinée à un seul domaine.

L'intégrité référentielle est donc assurée **au niveau applicatif** : **registrations-service** vérifie par appel REST que l'événement et le participant existent avant de créer une inscription.

2.4 Communication inter-services

Toute la communication passe par des **API REST synchrones** en JSON.

Inscription d'un participant

```
POST /registrations {eventId, participantId}
├── GET events-service/events/:id (l'événement existe ? capacité ?)
├── GET participants-service/participants/:id (le participant existe ?)
├── SELECT COUNT(*) ... status='CONFIRMED' (base locale : places prises)
├── capacité atteinte ? → 409 Conflict "Evenement complet"
├── déjà inscrit ? → 409 Conflict "deja inscrit"
└── sinon → 201 Created
```

Places restantes d'un événement

```
GET events-service/events/:id/availability
└── GET registrations-service/registrations/stats/events/:id
    puis remaining = capacity - confirmed
```

Les deux services s'appellent mutuellement, mais **jamais sur la même requête** : il n'y a donc pas de cycle d'appel. `events-service` interroge `registrations-service` uniquement sur l'endpoint de disponibilité, et `registrations-service` interroge `events-service` uniquement pour lire un événement.

Chaque appel sortant est protégé par un **délai d'expiration** (`AbortController`, 3 secondes par défaut) afin qu'un service lent ne bloque jamais l'appelant.

2.5 Gestion des pannes

Situation	Comportement
Service distant injoignable lors d'une écriture	<code>503</code> avec un message explicite ; aucune inscription n'est créée
Service distant injoignable lors d'une lecture enrichie	La liste est renvoyée sans l'information manquante (<code>participant: null</code>)
PostgreSQL indisponible au démarrage	15 tentatives espacées de 2 s avant abandon
PostgreSQL indisponible en fonctionnement	<code>/ready</code> renvoie <code>503</code> , ce qui permet à l'orchestrateur de réagir

2.6 Passerelle API

Le frontend n'appelle jamais un port de service directement : il appelle `/api/<ressource>`, et Nginx (en production) ou Vite (en développement) route la requête vers le bon conteneur.

Préfixe appelé par le navigateur	Destination
<code>/api/events/*</code>	<code>events-service:3001/events/*</code>
<code>/api/participants/*</code>	<code>participants-service:3002/participants/*</code>
<code>/api/registrations/*</code>	<code>registrations-service:3003/registrations/*</code>
<code>/api/health/<service></code>	<code><service>/health</code>

Ce point d'entrée unique supprime tout problème de CORS, masque la topologie interne et permet de changer les ports sans toucher au code du frontend.

2.7 Architecture interne d'un service

Chaque microservice est organisé en couches, avec **injection de dépendances** à la racine :

```
index.js      bootstrap : pool PostgreSQL, migrations, écoute, arrêt propre
├── app.js     application Express (middlewares, sondes, routeurs, erreurs)
├── routes/    validation du payload, codes HTTP
├── services/  logique métier (registrations-service)
├── repositories/ accès aux données
│   ├── postgres-...-repository.js utilisé en production
│   └── in-memory-...-repository.js utilisé par les tests
```

`app.js` reçoit son dépôt en paramètre. C'est ce qui permet aux tests de lancer l'application **complète** (middlewares, routage, gestion d'erreurs) avec un dépôt en mémoire, donc sans PostgreSQL.

3. Description des microservices

3.1 events-service (port 3001)

Modèle de données : table `events`

Colonne	Type	Contraintes
<code>id</code>	<code>SERIAL</code>	clé primaire
<code>title</code>	<code>VARCHAR(200)</code>	non nul
<code>description</code>	<code>TEXT</code>	facultatif
<code>event_date</code>	<code>TIMESTAMPZ</code>	non nul, indexé
<code>location</code>	<code>VARCHAR(200)</code>	non nul, indexé
<code>capacity</code>	<code>INTEGER</code>	non nul, <code>CHECK (capacity > 0)</code>
<code>created_at</code> / <code>updated_at</code>	<code>TIMESTAMPZ</code>	<code>DEFAULT NOW()</code>

Endpoints

Méthode	Endpoint	Description	Codes
<code>POST</code>	<code>/events</code>	Créer un événement	201, 400
<code>GET</code>	<code>/events</code>	Lister (<code>?date</code> , <code>?location</code> , <code>?search</code> , <code>?from</code> , <code>?to</code>)	200
<code>GET</code>	<code>/events/:id</code>	Détail	200, 404
<code>GET</code>	<code>/events/:id/availability</code>	Places restantes	200, 404, 503
<code>PUT</code>	<code>/events/:id</code>	Modifier (partiel accepté)	200, 400, 404
<code>DELETE</code>	<code>/events/:id</code>	Supprimer	204, 404
<code>GET</code>	<code>/health</code> , <code>/ready</code>	Sondes	200, 503

Règles de validation : titre obligatoire (≤ 200 caractères), date ISO valide, lieu obligatoire, capacité entière strictement positive. Les erreurs sont renvoyées sous forme de liste, ce qui permet à l'interface de toutes les afficher d'un coup.

3.2 participants-service (port 3002)

Modèle de données : table `participants`

Colonne	Type	Contraintes
<code>id</code>	<code>SERIAL</code>	clé primaire
<code>full_name</code>	<code>VARCHAR(150)</code>	non nul, indexé
<code>email</code>	<code>VARCHAR(150)</code>	non nul, unique
<code>phone</code>	<code>VARCHAR(30)</code>	facultatif
<code>type</code>	<code>VARCHAR(20)</code>	<code>CHECK IN ('etudiant', 'professeur', 'externe')</code>
<code>created_at</code> / <code>updated_at</code>	<code>TIMESTAMPZ</code>	<code>DEFAULT NOW()</code>

Endpoints

Méthode	Endpoint	Description	Codes
<code>POST</code>	<code>/participants</code>	Créer un compte	201, 400, 409
<code>GET</code>	<code>/participants</code>	Lister (<code>?type</code> , <code>?search</code> , <code>?email</code> , <code>?name</code>)	200
<code>GET</code>	<code>/participants/search</code>	Recherche par <code>?email</code> ou <code>?name</code>	200, 400
<code>GET</code>	<code>/participants/:id</code>	Détail	200, 404
<code>PUT</code>	<code>/participants/:id</code>	Modifier le profil	200, 400, 404, 409
<code>DELETE</code>	<code>/participants/:id</code>	Supprimer	204, 404

Points d'attention : l'email est normalisé (minuscules, espaces retirés) avant insertion, et l'unicité est garantie à la fois par la contrainte SQL et par une traduction du code d'erreur PostgreSQL `23505` en réponse `409` .

3.3 registrations-service (port 3003)

Modèle de données : table `registrations`

Colonne	Type	Contraintes
id	SERIAL	clé primaire
event_id	INTEGER	non nul, indexé (pas de clé étrangère : autre base)
participant_id	INTEGER	non nul, indexé
status	VARCHAR(20)	CHECK IN ('CONFIRMED', 'CANCELLED'), défaut CONFIRMED
registered_at	TIMESTAMPZ	DEFAULT NOW()
cancelled_at	TIMESTAMPZ	renseigné à l'annulation

Un **index unique partiel** garantit qu'un participant ne peut avoir qu'une seule inscription active par événement, tout en autorisant la réinscription après annulation :

```
CREATE UNIQUE INDEX uniq_registration_confirmed
ON registrations (event_id, participant_id)
WHERE status = 'CONFIRMED';
```

Endpoints

Méthode	Endpoint	Description	Codes
POST	/registrations	Inscrire un participant	201, 400, 404, 409, 503
DELETE	/registrations/:id	Annuler (soft delete)	200, 404, 409
GET	/registrations	Lister (?eventId , ?participantId , ?status)	200, 400
GET	/registrations/:id	Détail	200, 404
GET	/registrations/stats	Statistiques globales + par événement	200
GET	/registrations/stats/events/:id	Compteurs d'un événement	200
GET	/registrations/availability/:id	Places disponibles	200, 404, 503
GET	/registrations/by-event/:id	Inscrits d'un événement (profils enrichis)	200, 404
GET	/registrations/by-participant/:id	Événements d'un participant	200, 404
GET	/events/:id/registrations	Alias REST du précédent	200, 404
GET	/participants/:id/events	Alias REST du précédent	200, 404

Annulation en *soft delete*. Annuler une inscription ne supprime pas la ligne : le statut passe à **CANCELLED** et **cancelled_at** est horodaté. Cela préserve l'historique, utile pour mesurer le taux d'annulation, tout en libérant immédiatement une place, puisque seuls les enregistrements **CONFIRMED** sont comptés.

Statistiques. L'endpoint **/registrations/stats** agrège les compteurs locaux puis interroge **events-service** pour ajouter le titre, la capacité et le taux de remplissage de chaque événement.

3.4 Conventions communes

- Réponses de liste normalisées : { "count": n, "data": [...] }
- Erreurs normalisées : { "error": "message", "details": ["..."] }
- snake_case** en base, **camelCase** dans les API (conversion dans les dépôts)
- Sondes **/health** (le processus répond) et **/ready** (la base répond)
- Arrêt propre sur **SIGTERM** : fermeture du serveur puis du pool PostgreSQL

4. Dockerisation

4.1 Dockerfile des microservices

Les trois services backend partagent le même Dockerfile, à la variable de port près. Il repose sur un **build multi-étapes** :

```
# Étape 1 : dépendances de production uniquement
FROM node:22-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev

# Étape 2 : image finale
FROM node:22-alpine AS runtime
ENV NODE_ENV=production PORT=3001
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY package.json ./
COPY src ./src
USER node
EXPOSE 3001
HEALTHCHECK --interval=30s --timeout=5s --start-period=20s --retries=3 \
  CMD wget --quiet --tries=1 --spider http://127.0.0.1:3001/health || exit 1
CMD ["node", "src/index.js"]
```

Choix effectués et justification :

Choix	Raison
<code>node:22-alpine</code>	~50 Mo contre ~380 Mo pour <code>node:22</code> ; Node 22 est la version LTS active
Build multi-étapes	Le cache npm et les outils d'installation restent dans l'étape <code>deps</code>
<code>COPY package*.json</code> avant <code>COPY src</code>	La couche <code>npm ci</code> n'est reconstruite que si les dépendances changent
<code>npm ci --omit=dev</code>	Installation reproductible depuis le lock file, sans dépendances de développement
<code>USER node</code>	Le conteneur ne tourne pas en <code>root</code> : limite l'impact d'une compromission
<code>ENV</code> pour la configuration	Aucune valeur d'environnement codée en dur (12-factor app)
<code>HEALTHCHECK</code>	Permet à Compose d'attendre qu'un service soit réellement prêt
<code>.dockerignore</code>	Exclut <code>node_modules</code> , <code>tests</code> , <code>.env</code> : contexte de build réduit et pas de secret embarqué

4.2 Dockerfile du frontend

Le frontend utilise un build multi-étapes encore plus marqué : Node sert uniquement à produire le bundle, et **n'est pas présent dans l'image finale**.

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build          # produit /app/dist

FROM nginx:1.27-alpine AS runtime
COPY nginx.conf            /etc/nginx/conf.d/default.conf
COPY proxy_headers.conf   /etc/nginx/proxy_headers.conf
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

L'image finale ne contient que Nginx et une trentaine de kilo-octets de fichiers statiques : environ **25 Mo**, contre plus de 400 Mo si l'on avait servi l'application avec Node.

Nginx remplit deux rôles : servir la *single-page application* (toute URL inconnue renvoie `index.html`, pour que le routage React fonctionne au rafraîchissement) et faire office de **passerelle API**.

4.3 Orchestration avec Docker Compose (bonus)

`docker-compose.yml` décrit les cinq conteneurs :

Service	Image	Port hôte	Dépendances
postgres	postgres:16-alpine	5432	:
events-service	build local	3001	postgres (healthy)
participants-service	build local	3002	postgres (healthy)
registrations-service	build local	3003	postgres, les 2 services (healthy)
frontend	build local	8080	les 3 services (healthy)

Éléments notables :

- **condition: service_healthy**, les services n'attendent pas seulement que PostgreSQL soit *démarré*, mais qu'il réponde à `pg_isready`. C'est ce qui supprime les échecs de démarrage aléatoires.
- **Volume nommé postgres-data**, les données survivent à `docker compose down` ; seul `down -v` les supprime.
- **Script d'initialisation**, `db/init/01-create-databases.sql` est monté dans `/docker-entrypoint-initdb.d` et crée les trois bases au tout premier démarrage.
- **Réseau eventhub-net**, les conteneurs se joignent par leur nom de service (`http://events-service:3001`) grâce au DNS interne de Docker.
- **Variables paramétrables**, ports, identifiants et registre sont lus depuis un fichier `.env` (`.env.example` fourni comme modèle).

```
docker compose up -d --build    # démarre toute la plateforme
docker compose ps              # état et santé des conteneurs
docker compose logs -f <service> # journaux
docker compose down -v         # arrêt et suppression des volumes
```

4.4 Vérification de la stack

L'ensemble a été démarré et mesuré sur un poste Windows 11 avec Docker Engine 29.6.2 et Docker Compose v5.3.1 (moteur WSL2).

Taille des images produites

Image	Taille	Base
eventhub/frontend	73,9 Mo	nginx:1.27-alpine
eventhub/events-service	239 Mo	node:22-alpine
eventhub/participants-service	238 Mo	node:22-alpine
eventhub/registrations-service	239 Mo	node:22-alpine
postgres:16-alpine	420 Mo	image officielle

Le frontend illustre l'intérêt du build multi-étapes : Node sert uniquement à produire le bundle et disparaît de l'image finale, qui ne contient que Nginx et les fichiers statiques.

Séquence de démarrage observée

Les journaux de `docker compose up` confirment que l'ordonnancement par `condition: service_healthy` est respecté :

```
Container eventhub-postgres    Healthy
Container eventhub-events     Started
Container eventhub-participants Started
Container eventhub-participants Healthy
Container eventhub-events     Healthy
Container eventhub-registrations Started
Container eventhub-registrations Healthy
Container eventhub-frontend    Started
```

`registrations-service` n'a démarré qu'une fois les deux services dont il dépend déclarés sains, et le frontend en dernier. Les cinq conteneurs sont `healthy` en une trentaine de secondes.

Isolation des bases

Le script `db/init/01-create-databases.sql` s'exécute au premier démarrage, et chaque service crée sa propre table :

Base	Tables
eventhub_events	events
eventhub_participants	participants
eventhub_registrations	registrations

Contrôles fonctionnels effectués via la passerelle Nginx (port 8080)

Scénario	Résultat attendu	Obtenu
Création d'un événement de 2 places	201	201
Inscriptions 1 et 2	201 , places restantes 1 puis 0	conforme
Inscription au-delà de la capacité	409	409 Evenement complet : 2/2 places occupees
Réinscription d'un participant déjà inscrit	409	409 Le participant 1 est deja inscrit a l evenement 5
Inscription d'un participant inexistant	404	404 Participant 9999 introuvable
Payload invalide	400 avec le détail des erreurs	4 erreurs listées
Annulation puis réinscription	place libérée, 201	conforme
URL profonde /inscriptions	index.html (routage SPA)	200

Le contrôle de doublon s'exécute avant le contrôle de capacité : un participant déjà inscrit à un événement complet reçoit le message « déjà inscrit », et non « complet ».

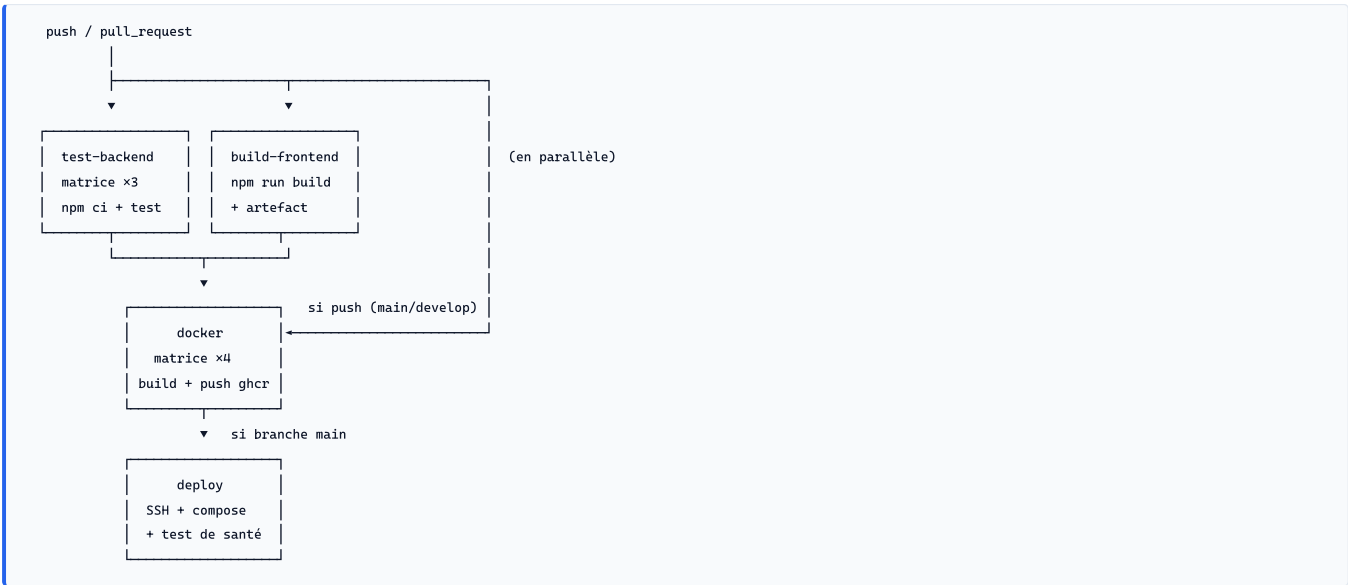
Persistence

Après `docker compose down` suivi de `docker compose up -d`, les données sont intactes : le volume nommé `postgres-data` survit à la suppression des conteneurs. Seul `docker compose down -v` les efface, ce qui déclenche à nouveau le script d'initialisation au démarrage suivant.

5. CI/CD avec GitHub Actions

5.1 Vue d'ensemble du pipeline

Fichier : `.github/workflows/ci-cd.yml`



5.2 Les sept étapes demandées

#	Étape	Réalisation
1	Checkout	<code>actions/checkout@v7</code> dans chaque job
2	Setup	<code>actions/setup-node@v7</code> , Node 22, <code>cache: npm</code> avec <code>cache-dependency-path</code> par service
3	Tests	<code>npm test</code> : tests unitaires des 3 services, exécutés en parallèle via une matrice
4	Build	<code>npm ci</code> pour chaque service ; <code>npm run build</code> (Vite) pour le frontend, bundle publié en artefact
5	Docker Build	<code>docker/build-push-action@v7</code> avec Buildx, matrice sur les 4 composants
6	Docker Push	Publication sur GitHub Container Registry (<code>ghcr.io</code>), authentification via le <code>GITHUB_TOKEN</code> intégré
7	Deploy	Connexion SSH au serveur, <code>docker compose pull && up -d</code> , puis vérification de <code>/health</code>

5.3 Actions utilisées

Action	Rôle
<code>actions/checkout@v7</code>	Récupération du code source
<code>actions/setup-node@v7</code>	Installation de Node.js et cache des dépendances npm
<code>actions/upload-artifact@v7</code>	Conservation du bundle frontend (7 jours)
<code>docker/setup-buildx-action@v4</code>	Moteur de build avancé (cache, multi-plateforme)
<code>docker/login-action@v4</code>	Authentification sur <code>ghcr.io</code>
<code>docker/metadata-action@v6</code>	Génération automatique des tags et labels OCI
<code>docker/build-push-action@v7</code>	Construction et publication des images
<code>appleboy/ssh-action@v1.2.5</code>	Exécution des commandes de déploiement sur le serveur

5.4 Optimisations mises en place

- Matrices de parallélisation.** Les trois suites de tests s'exécutent simultanément, de même que les quatre constructions d'images. Le temps total du pipeline correspond à celui du job le plus lent, pas à leur somme.
- `fail-fast: false`.** Si `events-service` échoue, les autres continuent : on obtient tous les diagnostics en une seule exécution.
- Cache npm** (`cache: npm`) et **cache de couches Docker** (`cache-from/to: type=gha`), avec un `scope` distinct par service pour éviter que les caches ne s'écrasent mutuellement.
- concurrency avec `cancel-in-progress`.** Un nouveau `push` sur une branche annule l'exécution précédente, ce qui économise des minutes de CI.
- Étiquetage des images** par `metadata-action` : nom de branche, SHA court, et `latest` uniquement sur la branche par défaut. Chaque déploiement est donc traçable jusqu'au commit exact.

5.5 Déclencheurs et conditions

Événement	Jobs exécutés
Pull Request vers <code>main</code> ou <code>develop</code>	<code>test-backend</code> , <code>build-frontend</code>
Push sur <code>develop</code>	+ <code>docker</code> (images taguées <code>develop</code>)
Push sur <code>main</code>	+ <code>docker</code> (tag <code>latest</code>) + <code>deploy</code>
<code>workflow_dispatch</code>	Déclenchement manuel

Le job `deploy` vérifie d'abord la présence des secrets `DEPLOY_HOST` , `DEPLOY_USER` et `DEPLOY_SSH_KEY` . S'ils sont absents, il affiche la marche à suivre et se termine en succès : **le pipeline n'échoue pas** parce qu'aucun serveur n'a encore été provisionné, ce qui serait un faux négatif gênant.

5.6 Stratégie de branches

Branche	Rôle	Effet en CI
<code>main</code>	Production	Tests + images + déploiement automatique
<code>develop</code>	Intégration continue	Tests + images taguées <code>develop</code>
<code>feature/*</code>	Nouvelle fonctionnalité	Tests via Pull Request vers <code>develop</code>
<code>fix/*</code>	Correction de bug	Tests via Pull Request

Le dépôt fournit un gabarit de Pull Request (`.github/pull_request_template.md`) qui rappelle les vérifications attendues : tests passants, build Docker fonctionnel, README à jour, aucun secret commité.

6. Interface frontend

6.1 Technologies et parti pris graphique

L'interface est une *single-page application* **React 18** construite avec **Vite 5** et routée par **React Router 6**. Le style est écrit en CSS natif (variables CSS, Grid, Flexbox), sans framework, ce qui garde le bundle léger (environ 60 Ko compressés) et évite toute dépendance supplémentaire dans l'image Docker.

Le parti pris graphique est celui d'un outil de travail, pas d'une vitrine :

- **Filets plutôt qu'ombres portées.** Les blocs sont délimités par un trait d'un pixel. Aucune ombre, aucun dégradé.
- **Angles nets** (rayon de 2 px) appliqués de façon uniforme aux blocs, aux champs et aux boutons.
- **Une seule couleur d'accent**, un vert profond utilisé pour l'onglet actif, le bouton principal et les jauges. Le rouge brique est réservé aux actions destructrices et à l'état « complet », donc porteur de sens.
- **Neutres légèrement chauds** plutôt qu'un gris bleuté, moins clinique à l'écran.
- **Chiffres en chasse fixe et en tabulaire** (`font-variant-numeric: tabular-nums`) pour que les colonnes s'alignent verticalement.
- **Aucune icône décorative.** La navigation et les états sont exprimés par le texte, ce qui évite un jeu d'icônes à maintenir et reste lisible en noir et blanc à l'impression.

La navigation est un bandeau horizontal en haut de page ; l'onglet actif est signalé par un filet vert sous le libellé.

Toutes les requêtes passent par un client HTTP unique (`src/api/client.js`) qui centralise l'URL de base, la sérialisation JSON et la traduction des erreurs de l'API en messages affichables.

6.2 Les quatre écrans

Tableau de bord. Vue consolidée agrégeant les trois microservices en parallèle : nombre d'événements, de participants, d'inscriptions confirmées et taux de remplissage global. Ces quatre chiffres sont posés côte à côte, séparés par un filet, sans conteneur décoratif. Un tableau détaille ensuite le remplissage par événement, une liste donne l'état de santé des trois services et une autre les prochaines échéances.

Événements. Liste en lignes séparées par un filet, chacune donnant le titre, la description, la date, le lieu, les places restantes et une jauge de remplissage. La mention « Complet » et la jauge rouge signalent un événement saturé. Filtres par recherche plein texte, par lieu et par date. Création et modification via une fenêtre modale, suppression avec confirmation.

Participants. Tableau listant nom, adresse électronique, téléphone et profil. Recherche par nom ou adresse, filtre par profil. Le bouton « Inscriptions » ouvre une fenêtre modale qui interroge `registrations-service` pour afficher tous les événements auxquels ce participant est inscrit, avec le statut de chaque inscription.

Inscriptions. Sélection d'un événement et d'un participant. La disponibilité est rechargée à chaque changement d'événement et affichée en clair dans l'en-tête du bloc ainsi que sous forme de jauge. Le bouton se désactive et affiche « Événement complet » lorsqu'il n'y a plus de place. En dessous, la liste des inscrits avec leur statut et un bouton d'annulation qui libère immédiatement une place.

6.3 Détails d'ergonomie

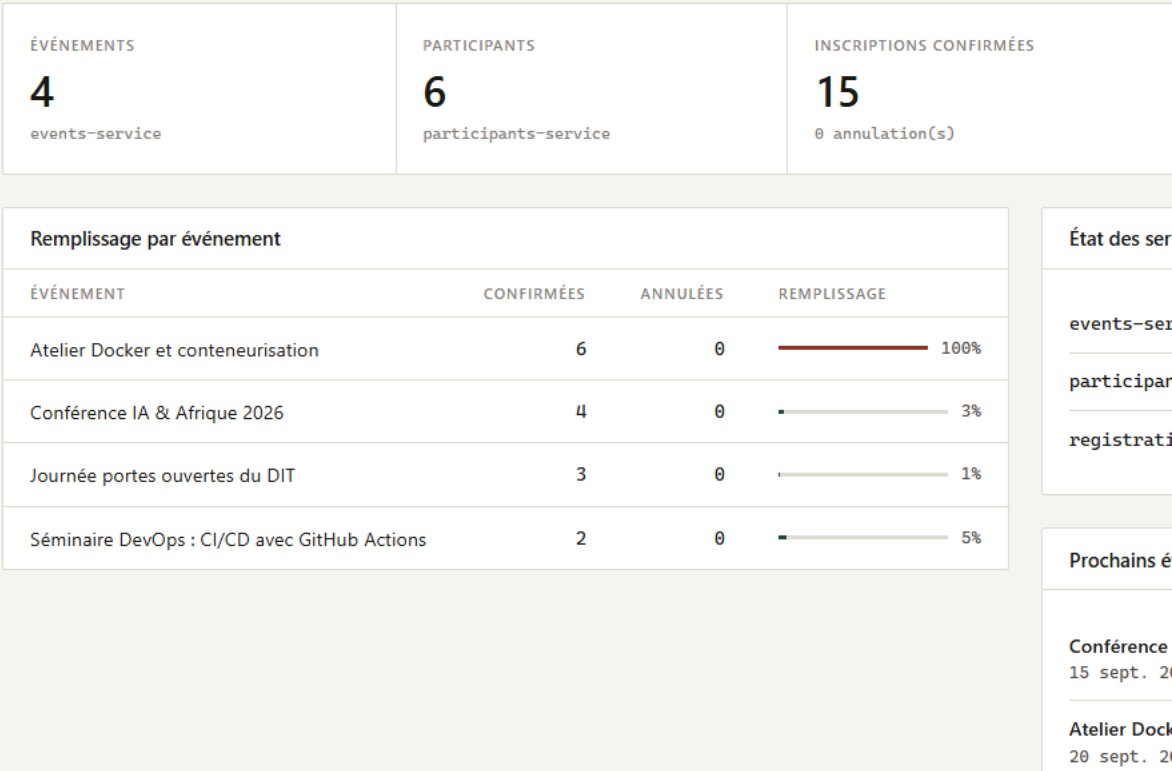
- **États explicites** : chargement, liste vide (message d'invitation à l'action), erreur (bandeau refermable), succès.
- **Messages d'erreur métier remontés tels quels** : « Evenement complet : 4/4 places occupees », « Un participant utilise deja l email ... ». L'API produisant des messages clairs, l'interface n'a pas à les réécrire.
- **Confirmation avant toute suppression** ou annulation.
- **Interface adaptative** : les onglets défilent horizontalement sous 640 px, le bandeau de chiffres passe sur une colonne, les tableaux défilent horizontalement.
- **Accessibilité** : fenêtres modales avec `role="dialog"` et fermeture à la touche `Échap`, libellés associés aux champs, contrastes conformes, animations désactivées sous `prefers-reduced-motion`.

6.4 Captures d'écran

Figure 1 : Tableau de bord

Tableau de bord

Vue consolidée des événements, des participants et des inscriptions



Les quatre indicateurs du bandeau proviennent chacun d'un service différent, agrégés en parallèle par le frontend. Le tableau « Remplissage par événement » combine les compteurs de `registrations-service` avec les titres et capacités de `events-service` ; l'atelier Docker y apparaît à 100 %. La liste de droite interroge les sondes `/health` des trois services.

Figure 2 : Liste des événements

Événements

Création, modification et suivi de la capacité

Rechercher un titre ou une description	Filtrer par lieu	jj/mm/aaaa
Conférence IA & Afrique 2026 Conférence annuelle du DIT sur les applications de l'intelligence artificielle en Afrique de l'Ouest.	15 sept. 2026, 09:00 Amphithéâtre A, DIT Dakar	116 place(s) r
Atelier Docker et conteneurisation Atelier pratique : images, volumes, réseaux et docker-compose.	20 sept. 2026, 14:00 Salle B, DIT Dakar	Complet
Séminaire DevOps : CI/CD avec GitHub Actions Automatisation des tests, du build et du déploiement continu.	01 oct. 2026, 10:00 Laboratoire informatique, DIT Dakar	38 place(s) re
Journée portes ouvertes du DIT Présentation des filières et des projets étudiants.	12 oct. 2026, 08:30 Campus DIT, Dakar	297 place(s) 1

Architecture microservices : events-service, participants-service, registrations-service.

Chaque ligne affiche les places restantes obtenues auprès de `registrations-service`. L'atelier Docker, dont les 6 places sont prises, porte la mention « Complet » et une jauge rouge, tandis que les autres événements affichent leur nombre de places disponibles. La barre de filtres permet une recherche plein texte, un filtre par lieu et un filtre par date.

Figure 3 : Gestion des participants

Participants

Comptes étudiants, professeurs et intervenants externes

Rechercher par nom ou adresse électronique			Tous les profils	
NOM COMPLET	ADRESSE ÉLECTRONIQUE	TÉLÉPHONE	PROFIL	ACTIONS
Awa Diop	awa.diop@dit.sn	+221 77 123 45 67	ÉTUDIANT	
Cheikh Sy	cheikh.sy@dit.sn	+221 76 111 22 33	PROFESSEUR	
Coumba Ba	coumba.ba@dit.sn	+221 77 444 55 66	ÉTUDIANT	
Fatou Sall	fatou.sall@externe.com	+221 70 555 11 22	EXTERNE	
Ibrahima Fall	ibrahima.fall@dit.sn	+221 78 222 33 44	ÉTUDIANT	
Moussa Ndiaye	moussa.ndiaye@dit.sn	+221 76 987 65 43	PROFESSEUR	

Le profil de chaque participant (étudiant, professeur, externe) est indiqué dans une étiquette cadrée par un filet. Le bouton « Inscriptions » ouvre une fenêtre modale qui interroge `registrations-service` pour afficher tous les événements auxquels ce participant est inscrit, avec le statut de chaque inscription.

Figure 4 : Inscriptions et disponibilité

Inscriptions

Inscription des participants et suivi des places disponibles

Nouvelle inscription

ÉVÉNEMENT

Conférence IA & Afrique 2026

PARTICIPANT

Sélectionner un participant

3%

Inscrits (4)

PARTICIPANT	ADRESSE ÉLECTRONIQUE	PROFIL	INSCRIT LE
Ibrahima Fall	ibrahima.fall@dit.sn	Étudiant	07 août 2026, 20:32
Fatou Sall	fatou.sall@externe.com	Externe	07 août 2026, 20:32
Moussa Ndiaye	moussa.ndiaye@dit.sn	Professeur	07 août 2026, 20:32
Awa Diop	awa.diop@dit.sn	Étudiant	07 août 2026, 20:32

La disponibilité de l'événement sélectionné est rechargée à chaque changement et rappelée en clair dans l'en-tête du bloc. Lorsque la capacité est atteinte, le bouton d'inscription se désactive et affiche « Événement complet » ; le contrôle est également appliqué côté serveur, qui répond `409 Conflict` si la requête est envoyée malgré tout. La liste des inscrits, enrichie du profil de chaque participant, permet d'annuler une inscription et de libérer immédiatement une place.

7. Difficultés rencontrées et solutions apportées

7.1 Dépendance circulaire apparente entre deux services

Problème. Le sujet demande une vérification des places restantes sur `events-service` et une détection des places disponibles sur `registrations-service`. Or `events-service` connaît la capacité mais pas les inscriptions, et `registrations-service` l'inverse. Chacun a besoin de l'autre.

Solutions envisagées. (a) Dupliquer un compteur `registered_count` dans la table `events`, mis à jour par `registrations-service`, rapide en lecture, mais introduit une donnée dénormalisée à maintenir cohérente. (b) Appels REST croisés.

Solution retenue. Les appels REST croisés, en s'assurant qu'ils ne se produisent jamais dans la même chaîne de requête : `events-service/events/:id/availability` appelle `registrations-service/registrations/stats/events/:id`, un endpoint qui lit uniquement sa base locale et n'effectue aucun appel sortant. Il n'y a donc pas de récursion possible. La donnée reste à un seul endroit, au prix d'un appel réseau supplémentaire (quelques millisecondes sur le réseau Docker).

7.2 Tests unitaires dépendant de PostgreSQL

Problème. Écrire des tests qui exigent une base de données rend la CI lente et fragile, et empêche de lancer `npm test` sur un poste sans PostgreSQL.

Solution. Le motif `repository` avec injection de dépendances. Chaque service définit deux implémentations du même contrat : `postgres-...-repository` en production et `in-memory-...-repository` dans les tests. Comme `createApp()` reçoit son dépôt en paramètre, les tests lancent l'application complète (middlewares, routage, validation, gestion d'erreurs) sur un port éphémère, et l'interrogent par de vraies requêtes HTTP. On teste donc le comportement réel de l'API, sans aucune infrastructure. Les 62 tests s'exécutent en quelques secondes.

Le dépôt en mémoire reproduit fidèlement les contraintes de la base (unicité de l'email, unicité de l'inscription confirmée) pour que les tests restent représentatifs.

7.3 Collision de routes dans la passerelle Nginx

Problème. `registrations-service` expose `/events/:id/registrations`, une URL REST naturelle. Mais côté passerelle, `/api/events/*` doit être routé vers `events-service`. L'URL `/api/events/1/registrations` aurait donc été envoyée au mauvais service.

Solution. Ajout d'alias dans l'espace de noms du service propriétaire : `/registrations/by-event/:id` et `/registrations/by-participant/:id`. Le frontend n'utilise que ces alias, ce qui permet à Nginx de n'avoir qu'un **seul préfixe par service**, une configuration simple et sans ambiguïté. Les routes REST d'origine restent exposées pour un usage direct de l'API.

7.4 Ordre de démarrage des conteneurs

Problème. Au premier `docker compose up`, les services démarraient avant que PostgreSQL n'accepte les connexions, et s'arrêtaient en erreur.

Solution. Deux mécanismes complémentaires :

1. `depends_on` avec `condition: service_healthy` et un `healthcheck pg_isready` sur le conteneur PostgreSQL, pour que Compose attende la disponibilité réelle de la base.
2. Une boucle de reconnexion applicative (`waitForDatabase` : 15 tentatives espacées de 2 secondes) dans chaque service, qui rend le démarrage robuste même en dehors de Compose, sur Kubernetes ou après un redémarrage de la base.

7.5 Conflit entre `/registrations/:id` et `/registrations/stats`

Problème. Express évalue les routes dans l'ordre de déclaration. Avec `/registrations/:id` déclarée en premier, l'appel `/registrations/stats` était capturé par la route paramétrée, et `parseId('stats')` renvoyait `400`.

Solution. Déclarer systématiquement les routes littérales (`/stats`, `/availability/:id`, `/by-event/:id`, `/search`) **avant** les routes paramétrées, avec un commentaire explicite dans le code pour éviter qu'une future modification ne réintroduise le problème. Des tests couvrent spécifiquement ces routes.

7.6 Fuseaux horaires et champs de formulaire

Problème. L'API stocke et renvoie des dates ISO en UTC, alors que `<input type="datetime-local">` attend une date locale sans fuseau. Éditer un événement décalait son horaire.

Solution. Deux fonctions de conversion explicites dans le frontend : `toInputDateTime()` retire le décalage local pour remplir le champ, et la soumission repasse par `new Date(...).toISOString()`. La base ne contient ainsi que de l'UTC (`TIMESTAMPZ`), et l'affichage se fait toujours dans le fuseau de l'utilisateur via `toLocaleString('fr-FR')`.

8. Améliorations possibles

8.1 Sécurité

- **Authentification et autorisation.** Aucune n'est implémentée : le sujet ne la demandait pas, mais en production il faudrait un service d'identité (JWT ou OpenID Connect) et une distinction entre organisateur et participant. Les services ne devraient accepter que des jetons signés, y compris entre eux (*mTLS* ou jeton de service).
- **Limitation de débit** (`express-rate-limit`) et en-têtes de sécurité (`helmet`) sur les endpoints publics.
- **Gestion des secrets.** Les mots de passe transitent aujourd'hui par des variables d'environnement Compose ; un coffre (Docker Secrets, HashiCorp Vault, GitHub Environments) serait préférable.
- **Analyse de vulnérabilités** dans la CI : `npm audit` , et scan des images avec Trivy avant publication.

8.2 Robustesse et résilience

- **Disjoncteur** (*circuit breaker*) sur les appels inter-services, pour cesser d'appeler un service en panne au lieu d'attendre le délai d'expiration à chaque requête.
- **Réservation atomique des places.** Entre le comptage et l'insertion, deux requêtes simultanées peuvent théoriquement dépasser la capacité. L'index unique empêche la double inscription du *même* participant, mais pas le dépassement par deux participants différents. Une transaction avec verrouillage, ou un compteur atomique, fermerait complètement la fenêtre.
- **Communication asynchrone.** Un bus d'événements (RabbitMQ, Kafka) pour les notifications supprimerait le couplage temporel entre services.

8.3 Fonctionnalités

- **Notifications par email** à l'inscription et rappel avant l'événement. Ce serait un quatrième microservice (`notifications-service`), consommateur d'événements.
- **Liste d'attente automatique** : inscrire les participants en attente lorsqu'une annulation libère une place.
- **Export CSV/PDF** des listes d'émargement.
- **QR code d'inscription** et pointage des présences le jour J.
- **Recherche avancée** et pagination des listes (nécessaire au-delà de quelques centaines d'enregistrements).

8.4 Qualité et observabilité

- **Tests d'intégration** avec Testcontainers, pour valider les dépôts PostgreSQL réels en complément des dépôts en mémoire.
- **Tests de bout en bout** de l'interface avec Playwright, intégrés à la CI.
- **Documentation d'API** OpenAPI/Swagger générée et publiée automatiquement.
- **Journalisation structurée** (JSON) avec identifiant de corrélation propagé entre services, agrégée dans une pile ELK ou Loki.
- **Métriques et traces** : exposition Prometheus, tableaux de bord Grafana, traçage distribué OpenTelemetry pour visualiser une inscription de bout en bout.
- **Analyse statique** : ESLint et Prettier appliqués dans la CI.

8.5 Déploiement

- **Kubernetes** pour la mise à l'échelle horizontale, avec des sondes `liveness` / `readiness` déjà prêtes (`/health` et `/ready` existent).
 - **Déploiement progressif** (*blue-green* ou *canary*) plutôt qu'un remplacement direct des conteneurs.
 - **Migrations de schéma versionnées** (Flyway, node-pg-migrate) au lieu du `CREATE TABLE IF NOT EXISTS` au démarrage, qui convient à un projet pédagogique mais ne gère pas les évolutions de schéma.
 - **Environnement de pré-production** alimenté par la branche `develop` , en miroir de la production.
-

Conclusion

EventHub met en œuvre l'ensemble des points demandés : une architecture microservices avec trois services backend communiquant en REST, une base relationnelle découpée par service, un frontend React, un **Dockerfile** par composant appliquant les bonnes pratiques, une orchestration Docker Compose, et un pipeline GitHub Actions couvrant les sept étapes du sujet, du *checkout* au déploiement.

Au-delà de la conformité au cahier des charges, le projet a été l'occasion de traiter des problèmes réels d'architecture distribuée : gestion d'une dépendance croisée entre services, ordonnancement du démarrage des conteneurs, testabilité sans infrastructure, et dégradation contrôlée en cas de panne partielle. Les sections « Difficultés » et « Améliorations » documentent aussi bien les compromis assumés que les limites qui subsistent.